Delhi Flood Intelligence Platform

An AI-assisted flood-risk analysis and scenario simulation platform for ward-level planning, operational response, and urban resilience.

Industry-level technical project report covering architecture, design, implementation, deployment, testing, and future roadmap.

Description

This document describes a flood-risk intelligence system built for urban authorities, emergency planners, and resilience teams. The platform combines data ingestion, machine learning inference, geospatial risk mapping, and a web dashboard for scenario-based flood forecasting in Delhi wards.

Author: Prepared by OpenAI Codex for CodeNovaXNamit

Date: 27 April 2026

2. Abstract

The Delhi Flood Intelligence Platform is a decision-support system designed to transform rainfall observations and scenario uploads into coordinate-level flood-risk predictions and ward-level operational insights. The system exposes a FastAPI backend for scenario processing, uses a lightweight ensemble of machine learning models for risk estimation, stores run history in a local SQLite database, and provides a Next.js dashboard for visualization, hotspot inspection, and scenario uploads. The project is structured for academic submission as well as interview discussion because it combines data engineering, geospatial analytics, backend services, frontend product design, and deployment engineering in a single coherent platform.

3. Problem Statement

Urban flood response typically suffers from fragmented data sources, delayed analysis, and limited ward-level visibility. Emergency teams often receive rainfall data, drainage context, and historical vulnerability indicators in separate silos. This slows down decision-making during high-rainfall events and makes it difficult to prioritize local response measures. The project addresses this gap by building a platform that standardizes rainfall inputs, interpolates them over a reference grid, enriches them with static spatial features, predicts proxy flood risk, and presents the result through an operational dashboard.

- Manual flood assessment is too slow for operational planning.
- Ward-level decision-makers need localized risk rather than city-wide averages.
- Multiple data formats make ingestion and analysis error-prone.
- Non-technical stakeholders need a visual workflow, not only CSV outputs.

4. Objectives

- Provide a repeatable pipeline for converting rainfall scenario CSV files into flood-risk predictions.
- Expose a clean API for scenario upload, run tracking, and result download.
- Offer a geospatial dashboard for ward visualization and hotspot analysis.
- Maintain a lightweight architecture that is easy to run locally or through Docker.
- Support viva, interview, and project-submission use cases with explainable system design.

5. System Overview

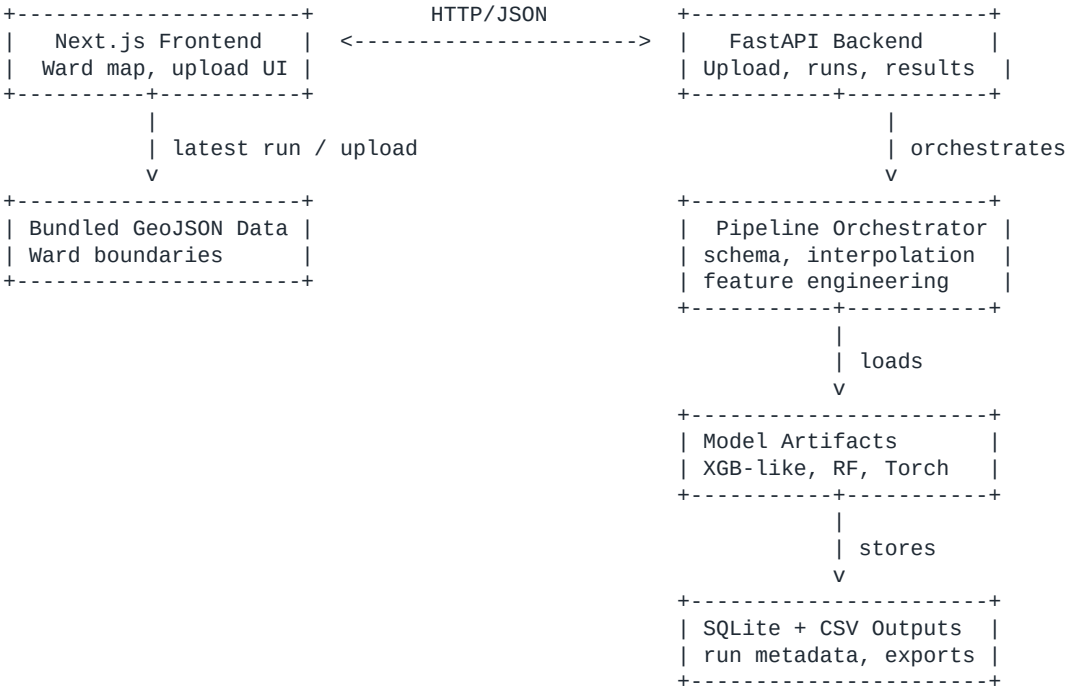
At a high level, the platform accepts rainfall observations or simulated rainfall scenarios, standardizes the schema, interpolates rainfall to a predefined model grid, generates inference features, merges static coordinate-level context, performs ensemble prediction, and stores artifacts for auditability. The frontend periodically queries the backend for the latest successful pipeline run and overlays risk outputs on top of ward or hotspot visualizations. This separation between presentation and computation keeps the system modular and production-friendly.

6. Architecture

6.1 Architectural Style

The implementation follows a modular monolith pattern with clearly separated frontend, backend, data-processing, and model layers. While not decomposed into independently deployed microservices, the system uses API boundaries and file/module boundaries that allow future service extraction if needed.

6.2 Component Diagram Explanation



6.3 Data Flow Explanation

- A user uploads a rainfall CSV from the dashboard or directly through the API.
- The backend stores the upload and creates a run record in SQLite.
- The pipeline normalizes columns such as date, lat/lon, and rainfall.
- Rainfall is interpolated to the reference model grid using inverse-distance weighting.
- Time-series and static geographic features are assembled for each grid coordinate.
- An ensemble model computes flood-risk probabilities.
- The latest date is extracted and transformed into coordinate-level risk points.
- Artifacts are written to CSV and the run summary is saved for retrieval by the frontend.

7. Technology Stack

Layer	Technology	Role in System
Frontend	Next.js 15, React 19, TypeScript, Tailwind CSS, Leaflet	Dashboard, geospatial visualization, scenario upload UI
Backend	Python 3.11+, FastAPI, Uvicorn	REST API, orchestration, artifact management
Database	SQLite	Stores pipeline run metadata and audit trail
Data Processing	Pandas, NumPy	Schema normalization, interpolation, feature preparation
ML Inference	Custom ensemble + PyTorch	Risk probability estimation
Containerization	Docker, Docker Compose	Consistent local and deployment runtime
External API	wttr.in	Live weather widget fallback for the frontend

8. Detailed Modules

8.1 Frontend Module

The frontend is responsible for user interaction, ward visualization, and pipeline upload workflows. The main page dynamically loads the map and panel components to avoid unnecessary server-side rendering for browser-specific libraries like Leaflet. It uses a `useFloodData` hook to manage weather sync, latest run fetch, and scenario submission. When a successful pipeline result is available, the frontend converts backend result points into map hotspots.

8.2 Backend Module

The backend provides health endpoints, run listing, latest run retrieval, run download, and scenario upload. Route handlers remain thin and delegate processing to the pipeline module. This keeps I/O, validation, and business logic clearly separated. The backend also initializes the SQLite schema on startup so the service is self-bootstrapping for local development.

8.3 Database Module

The database layer uses SQLite for simplicity and portability. A single `pipeline_runs` table stores run IDs, scenario names, input/output paths, timestamps, row counts, summary JSON, and error messages. This supports auditability and makes the latest successful result query straightforward.

8.4 AI / ML Module

The machine learning module is an ensemble consisting of a gradient-boosted stump model, a random stump forest, and a PyTorch multilayer perceptron. The ensemble computes per-coordinate risk and combines model outputs with fixed normalized weights. This hybrid approach balances interpretability, low runtime overhead, and predictive diversity.

9. Database Design

9.1 Schema Design

The persistent relational design is intentionally compact because this project stores processing history rather than transaction-heavy user data. Metadata is normalized enough for reporting, while the large numerical artifacts remain in CSV files inside the outputs directory.

Column	Type	Purpose
run_id	TEXT PRIMARY KEY	Unique identifier for a pipeline execution
scenario_name	TEXT	Human-readable scenario label
uploaded_filename	TEXT	Original CSV name
status	TEXT	processing / completed / failed
created_at	TEXT	Run creation timestamp
updated_at	TEXT	Last update timestamp
input_rows	INTEGER	Rows accepted after schema standardization
interpolated_rows	INTEGER	Rows after grid interpolation
output_rows	INTEGER	Rows in final risk output
latest_prediction_date	TEXT	Latest scenario date included in final output
upload_path	TEXT	Saved original input file
interpolated_path	TEXT	Intermediate interpolated CSV
enriched_path	TEXT	Feature-enriched CSV

Column	Type	Purpose
predictions_path	TEXT	Full prediction CSV
final_output_path	TEXT	Coordinate risk CSV for consumers
summary_json	TEXT	Serialized summary payload
error_message	TEXT	Failure details when processing fails

9.2 Relationships

- One pipeline run owns many generated artifact files, but those files are stored on disk rather than as child rows.
- The backend treats `run_id` as the primary key for both database lookup and artifact-folder naming.
- The frontend consumes run summaries and result points through API responses instead of querying the database directly.

10. API Design

Method	Endpoint	Purpose
GET	/	Service metadata and quick links
GET	/health	Health-check endpoint
GET	/api/pipeline/runs	List recent pipeline runs
GET	/api/pipeline/runs/latest	Get latest successful run
GET	/api/pipeline/runs/{run_id}	Get run details
GET	/api/pipeline/runs/{run_id}/download	Download final result CSV
POST	/api/pipeline/runs	Upload scenario CSV and execute pipeline

10.1 Example Upload Request

```
POST /api/pipeline/runs
Content-Type: multipart/form-data

form-data:
  file = sample_week_flood_rainfall.csv
  scenario_name = monsoon-simulation-week-1
```

10.2 Example Success Response

```
{
  "run": {
    "run_id": "8d3d5f74-2f27-4687-9dc8-1adf1ac9d521",
    "scenario_name": "monsoon-simulation-week-1",
    "status": "completed",
    "input_rows": 840,
    "interpolated_rows": 17500,
    "output_rows": 2500,
    "latest_prediction_date": "2026-07-14",
    "download_url": "/api/pipeline/runs/8d3d5f74-2f27-4687-9dc8-1adf1ac9d521/download",
    "summary": {
      "max_risk": 0.913,
      "mean_risk": 0.274,
      "high_risk_points": 124,
      "medium_risk_points": 418,
      "low_risk_points": 1958
    },
    "result_points": [
      {"lat": 28.7041, "lon": 77.1025, "risk": 0.9131}
```

```
    ]
  }
}
```

11. Workflow / Pipeline

- Step 1: The client uploads a CSV file through the dashboard or API.
- Step 2: The backend validates the file extension and generates a unique run ID.
- Step 3: The original file is written into `outputs/pipeline/uploads`.
- Step 4: The schema is standardized to canonical names such as `grid_latitude`, `grid_longitude`, and `precipitation_mm`.
- Step 5: For each date, rainfall is interpolated over the reference grid using inverse-distance weighting.
- Step 6: Rolling and lag features are generated for temporal context.
- Step 7: Static spatial features are merged from coordinate-level reference data.
- Step 8: The ensemble generates prediction probabilities and binary risk labels.
- Step 9: The latest scenario date is transformed into a coordinate risk CSV suitable for visualization.
- Step 10: Summary metrics and artifact paths are persisted, and the frontend consumes the result.

12. Implementation Details

12.1 Key Algorithm: Schema Standardization

```
def standardize_columns(df):
    rename_map = {}
    if "lat" in df.columns:
        rename_map["lat"] = "grid_latitude"
    if "lon" in df.columns:
        rename_map["lon"] = "grid_longitude"
    if "rainfall" in df.columns:
        rename_map["rainfall"] = "precipitation_mm"

    df = df.rename(columns=rename_map).copy()
    required = {"date", "grid_latitude", "grid_longitude", "precipitation_mm"}
    missing = required - set(df.columns)
    if missing:
        raise ValueError(f"Missing required columns: {sorted(missing)}")
    return df
```

12.2 Key Algorithm: Grid Interpolation

The interpolation routine computes weighted rainfall values for a target reference grid. Observed coordinates for each date are compared against the target grid, exact coordinate matches are copied directly, and non-exact locations are estimated through inverse-distance weighting. This makes the platform robust to sparse or irregular input coverage.

```
distances = np.sqrt(((target_coords[:, None, :] - observed_coords[None, :, :]) ** 2).sum(axis=2))
safe_dist = np.clip(distances[remaining], 1e-6, None)
inverse = 1.0 / np.square(safe_dist)
weights = inverse / inverse.sum(axis=1, keepdims=True)
weighted[remaining] = weights @ observed_rain
```

12.3 Key Algorithm: Ensemble Prediction

```
p_xgb = bundle["xgb_like"].predict_proba(X_raw)[:, 1]
p_rf = bundle["random_forest"].predict_proba(X_raw)[:, 1]
with torch.no_grad():
    logits = bundle["neural_net"](torch.tensor(X_scaled, dtype=torch.float32)).numpy()
p_nn = sigmoid(logits)

raw_weights = np.array([0.6, 0.3, 0.2], dtype=float)
```

```
weights = raw_weights / raw_weights.sum()
p_ensemble = weights[0] * p_xgb + weights[1] * p_rf + weights[2] * p_nn
```

13. UI / UX Description

- Top Bar: shows city readiness score and simulation state.
- Ward Search: allows quick focus on a specific ward from the map.
- Flood Map: renders ward polygons and hotspot overlays with interactive drill-down.
- Sidebar: displays weather, pipeline upload form, summary cards, and scenario controls.
- Hotspot Panel: reveals detailed information when a ward is selected.
- Scenario Upload Card: lets the user upload rainfall CSV files and immediately run prediction.

The interface design prioritizes operational readability. Color, typography, and layout are chosen to highlight risk levels and guide attention toward actionable information. The dashboard is designed for analysts and civic operators, not just developers, so important actions such as uploading a scenario or downloading a result are placed directly in the main control panel.

14. Deployment

The project supports both direct local execution and containerized deployment. The backend Docker image installs Python dependencies, copies the API, source code, and required processed model/reference assets, and serves the application with Uvicorn on port 8000. The frontend Docker image uses a multi-stage Node build and serves the compiled Next.js application on port 9002. Docker Compose orchestrates the two containers and mounts a named volume for backend outputs so generated artifacts survive container restarts.

```
# Local
uvicorn api.app:app --host 127.0.0.1 --port 8000
cd frontend && npm run dev

# Containerized
docker compose up --build
```

15. Testing

15.1 Unit Testing

- Validate schema normalization with valid and invalid CSV column combinations.
- Test helper functions such as slug creation and summary aggregation.
- Verify database CRUD operations for run creation and status updates.
- Check feature engineering outputs such as lag and rolling-sum columns.

15.2 Integration Testing

- Upload a sample scenario through the API and verify the generated run metadata.
- Confirm that latest-run retrieval returns the newest successful run.
- Validate result download and CSV existence checks.
- Exercise the frontend flow from upload to hotspot rendering using mock or local API responses.

16. Security Considerations

- Input validation ensures the upload endpoint only accepts CSV files.
- The backend does not execute uploaded content; files are treated as data only.

- CORS is restricted to expected local frontend origins in the current implementation.
- Artifact paths are generated server-side to prevent path traversal by user input.
- Sensitive secrets are minimal in the current local architecture; environment files should still be excluded from Git.

For a production-grade deployment, the next step would be to introduce authentication, signed upload/download access, request rate limiting, audit logging, and storage hardening for generated artifacts.

17. Scalability & Performance

- The modular monolith can later be split into dedicated upload, inference, and reporting services.
- Model artifacts are cached in memory using `lru_cache`, reducing repeated disk loads.
- Static frontend geo-data is bundled locally, reducing backend dependency for initial rendering.
- Outputs are written as files rather than bloating the database with large blobs.
- A future production version could replace SQLite with PostgreSQL and use object storage for artifacts.

The current implementation is appropriate for demonstrations, academic submission, local analysis, and moderate internal use. For higher concurrency or wider public deployment, queue-based asynchronous processing and distributed storage would be advisable.

18. Challenges & Solutions

Challenge	Impact	Solution
Irregular rainfall input schemas	Pipelines fail on inconsistent column names	Added standardization logic that accepts multiple aliases
Sparse coordinate coverage	Missing prediction coverage over the city grid	Applied inverse-distance interpolation to a fixed reference grid
Frontend/backend integration	Visualization may show stale or incompatible data	Defined a stable latest-run API contract with summary and points
Large local data footprint	Repository becomes too large for GitHub	Ignored raw data and kept only lightweight runtime artifacts in version co
Model compatibility across saved artifacts	Legacy pickled classes may fail to load	Implemented a legacy unpickler map for backward compatibility

19. Future Scope

- Add authenticated multi-user support for municipal departments or agencies.
- Replace mock ward-risk fallbacks with full backend-driven geospatial summaries.
- Integrate historical flood events and live sensor feeds for better calibration.
- Introduce asynchronous job processing with Celery, Redis, or a cloud queue.
- Add explainable-AI summaries showing which features most influenced local risk predictions.
- Deploy on a managed cloud stack with PostgreSQL, object storage, and monitoring dashboards.

20. Conclusion

The Delhi Flood Intelligence Platform is a strong end-to-end engineering project because it connects real data-handling concerns with practical product delivery. The system demonstrates backend API design, geospatial processing, ML inference, persistence, frontend visualization, and deployment readiness in one coherent workflow. It is suitable for viva examinations, interviews, and project submission because each major software engineering concern can be discussed clearly: problem definition, architecture, module design, API contracts, data modeling, security, testing, and future extensibility.

End of document.